

Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning

Siyan Zhao^{*1}, Devaansh Gupta^{*1}, Qinqing Zheng², Aditya Grover¹

¹UCLA, ²Meta AI

May 6th, 2025
ASAP Seminar

Motivation

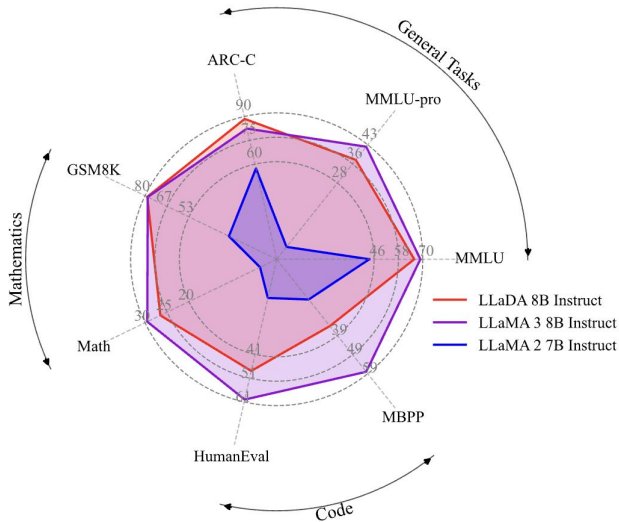
Recently, autoregressive (AR) LLMs benefit from RL post-training

- ❑ E.g. DeepSeek-R1, Kimi K1.5.
- ❑ Enables LLMs to generate and explore with reward signals rather than relying on static datasets.
- ❑ RL can incentivize/elicit sophisticated reasoning in LLMs.

However, these advances have primarily been limited to autoregressive LLMs that operate through **left-to-right, sequential inference**.

In parallel, diffusion large language models (dLLMs) have emerged as promising non-autoregressive alternatives for language modeling

TL;DR: We introduce LLaDA, a diffusion model with an unprecedented 8B scale, trained entirely from scratch, rivaling LLaMA3 8B in performance.



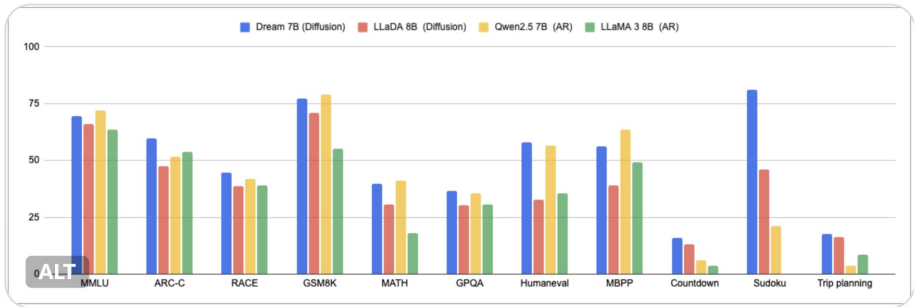
LLaDA 8B
Feb, 2025



Jiacheng Ye
@JiachengYe15



Excited to announce Dream 7B (Diffusion reasoning model): the most powerful open diffusion large language model to date.



Dream 7B
April, 2025

dLLMs generate through an iterative denoising process, in a coarse-to-fine manner.

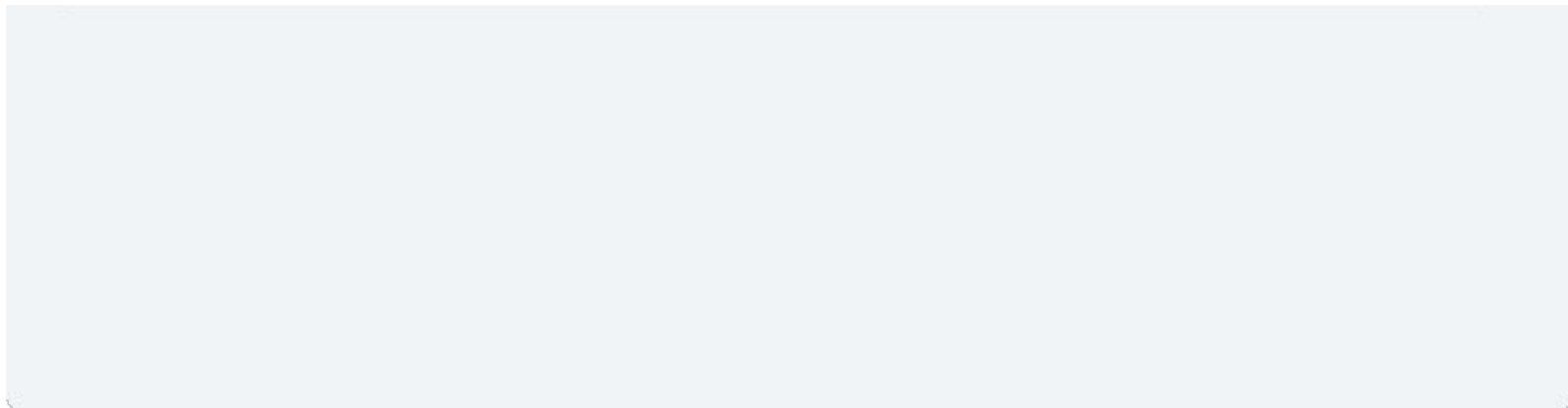


Figure: a generation example of Dream-7B-instruct.

Compared to AR LLMs, dLLMs brings benefits including:

- ❑ Parallel decoding -> Efficient Inference
- ❑ Bidirectional context -> Break Reversal Curse

	Model	Recipe	Pairs (M)	MMLU	MMLU-Pro	GSM8K	MATH	GPQA	
	Qwen2.5 7B	AR	SFT+RL	1+0.15	76.6	56.3	91.6	75.5	36.4
	LLaMA3 8B	AR	SFT+RL	-	68.4	41.9	78.3	29.6	31.9
	LLaDA 8B	Diffusion	SFT	4.5	65.5	37.0	78.6	26.6	31.8
	Dream 7B	Diffusion	SFT	1.8	67.0	43.3	81.0	39.2	33.0

(<https://hkunlp.github.io/blog/2025/dream/>)

Yet, no RL post-training has been applied to dLLMs

- ❑ **Our Goal:** Bring RL-based reasoning improvements to diffusion LLMs

Background - Diffusion Language Models

❑ Masked Diffusion Language Models.

- ❑ Add a [MASK] token into the vocabulary.
- ❑ Noise process: randomly replace tokens with [MASK].
- ❑ Example: SEDD Absorb, which uses an absorbing state transition matrix.
- ❑ Similar to BERT's masked LM approach

❑ Uniform Diffusion Language Models.

- ❑ Noise process: replace each token with any other vocabulary token, chosen uniformly at random.
- ❑ Example: SEDD Uniform

Background - Diffusion Language Models

- ❑ Prior works show that masked dLLMs consistently perform better than uniform dLLMs and scales better. (Lou et al., 2024; Campbell et al., 2024)
- ❑ Recent large dLLMs that scales $> 7B$, such as **DiffuLLaMA**, **LLaDA** and **Dream**, are all based on masked dLLMs.
- ❑ We focus on applying policy gradient RL to masked dLLMs in this work.

Background - Comparison of Masked dLLMs and BERT

- ❑ Both models have [MASK] token and train with CE loss.
- ❑ **Masking Schedule:**
 - ❑ BERT: fixed ~15% of tokens masked per example.
 - ❑ Masked dLLMs: varying masking rate over time.
- ❑ **Inference:**
 - ❑ BERT: single-step infilling—predict all [MASK] tokens in one forward pass.
 - ❑ Masked dLLMs: multi-step denoising—iteratively unmasking text tokens over diffusion steps.

Background - Masked Diffusion Large Language Models

- ❑ Masked dLLMs use a forward process that corrupts token sequences with masks over time $t \in [0, 1]$. (Sahoo et al., 2024; Shi et al., 2024; Austin et al., 2021; Ou et al., 2024; Zheng et al., 2024)
- ❑ At any time t , tokens remain unmasked with probability α_t (noise schedule), which strictly decreases as t increases.
- ❑ When $t = 1$, all tokens become masked.

Background - Masked Diffusion Large Language Models

Training Masked dLLMs, e.g. LLaDA:

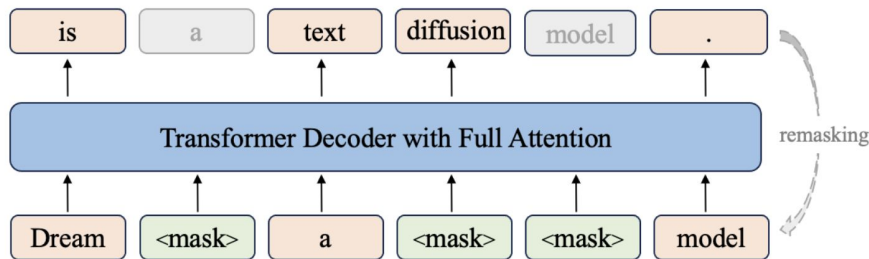
- ❑ Given a specific noise schedule: α_t ; In LLaDA: $\alpha_t = 1 - t$
- ❑ Using a bidirectional unmasking predictor f_θ (e.g. transformer).
- ❑ Sampling $t \in [0, 1)$, masking tokens accordingly, and predicting originals tokens and train with cross-entropy loss:

$$-\mathbb{E}_{t \sim \mathcal{U}[0,1), x_0 \sim p_{\text{data}}, x_t \sim q_{t|0}(x_t|x_0)} \left[\frac{1}{t} \sum_{k=1}^{|x_t|} \mathbb{1}[x_t^k = \text{mask}] \log f_\theta(x_0^k | x_t) \right]$$

Background - Masked Diffusion Large Language Models

Training Masked dLLMs: **Adapt from existing AR models.**

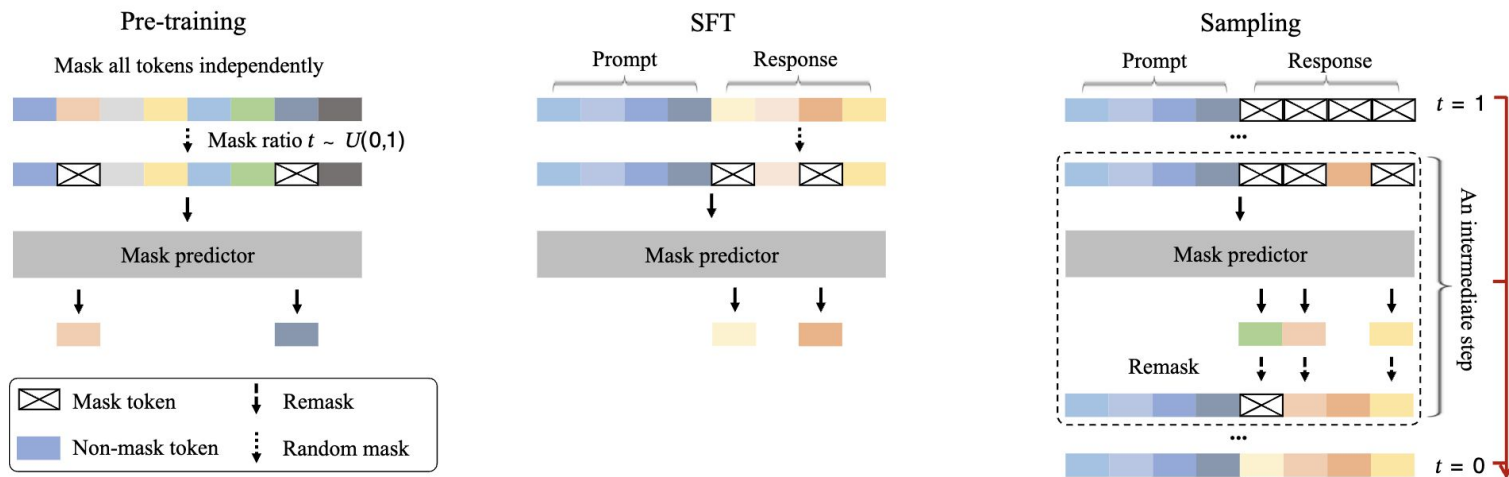
- ❑ DiffuLLaMA: adopted from Llama 2 7B.
- ❑ Dream: adopted from Qwen2.5 7B
- ❑ Predicts masked tokens in a **shifted manner**, allowing for weight initialization with AR models.
- ❑ Adapt causal attention mask to bidirectional mask.



(b) Diffusion Modeling in Dream

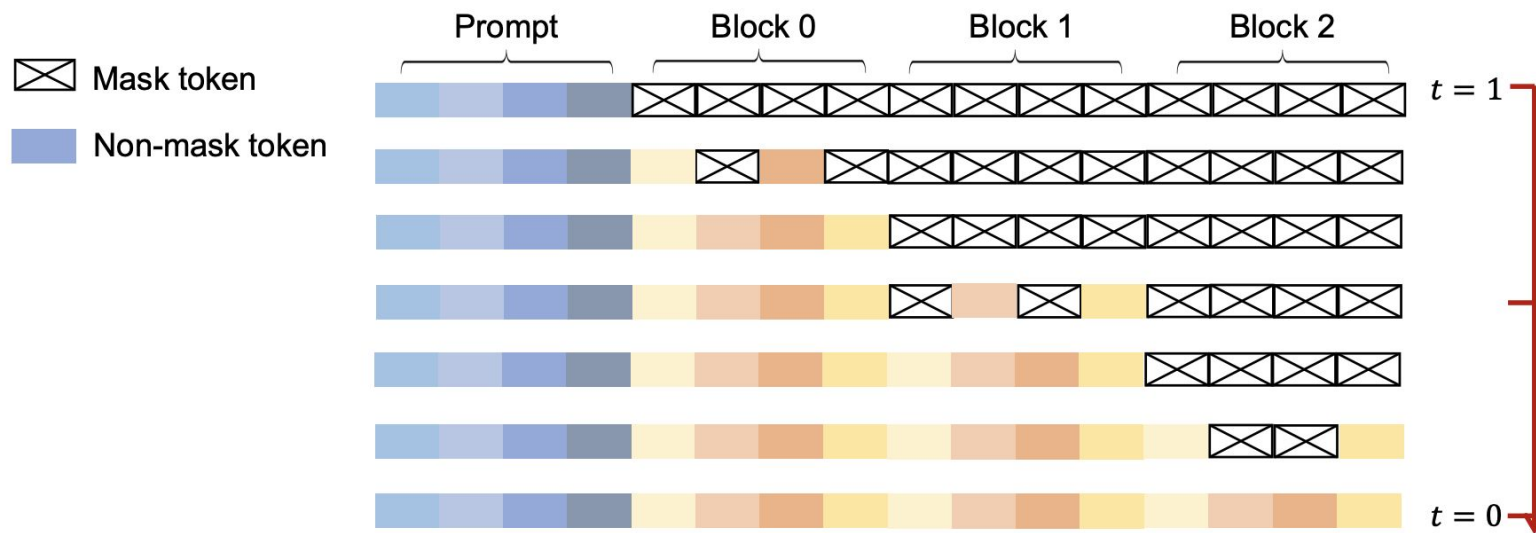
Background - Masked Diffusion Large Language Models

LLaDA scales up masked dLLMs to 8B from scratch.



(image from Llada (Nie et al., 2024))

Background - Semi-AR decoding in LLaDA.



(image from Llada (Nie et al., 2024))

Background - Policy Gradient Methods for LLMs

- ❑ Policy Gradient Methods (e.g., PPO) are widely used to fine-tune LLMs.
 - ❑ PPO requires a state value function to estimate advantages → high compute and memory.
- ❑ Group Relative Preference Optimization (GRPO)
 - ❑ GRPO avoids learning a value model.
 - ❑ For a query, sample G responses $\{o_1, o_2, \dots, o_G\}$ from policy.
 - ❑ Assign advantage to all tokens in responses using:

$$A_i^k(\pi) = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G)}$$

- ❑ GRPO objective: clipped ratio + KL penalty

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

Log-likelihood Computation for policy gradient optimization

The objective requires computing the (log-)likelihood ratio of π_θ and $\pi_{\theta_{old}}$, at both the token level (for the advantage weights) and the sequence level (for the KL term).

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_\theta || \pi_{ref}] \right\}$$

Log-likelihood Computation for policy gradient optimization

- ❑ AR LLMs directly model the exact per-token log-probabilities with CE loss
- ❑ The sequence-level log-probability can be easily computed through the chain rule using one forward pass:

$$\log \pi_{\text{AR}}(o|q) = \sum_{k=1}^{|o|} \log \pi_{\text{AR}}(o^k|q, o^{<k})$$

- ❑ However, dLLMs do not adhere to sequential factorization of the sequence log-probability as it is trained to maximize a variational lower bound.
- ❑ The exact marginal likelihood is intractable

For Sequence Log Probability: Mean-Field Approximation

- ❑ dLLMs treat the token sequence as a whole and its sequence-level log-probability lacks the AR decomposition.
- ❑ To efficiently estimate it, we use a simple mean-field decomposition that approximates:

$$\log \pi_{\theta}(o|q) \text{ by } \sum_{k=1}^{|o|} \log \pi_{\theta}(o^k|q)$$

- ❑ Approximate a complicated joint distribution over many variables with a product of simpler, independent distributions.

For Per-Token Log Probability:

- ❑ The per-token log-probability are also costly to compute since the decoding process invokes the unmasking predictor f_θ multiple times
- ❑ To efficiently estimate it, we use a simple one-step estimator:
 - ❑ We perturb q where every token is randomly masked out, resulting in q' .

$$\log f_\theta(o^k | q' \oplus \text{mask} \dots \oplus \text{mask})$$

- ❑ We then do one-step unmasking to obtain and use it as an estimation of

$$\log \pi_\theta(o^k | q), 1 \leq k \leq |o|.$$

MC type of approximation will cause excessive memory usage

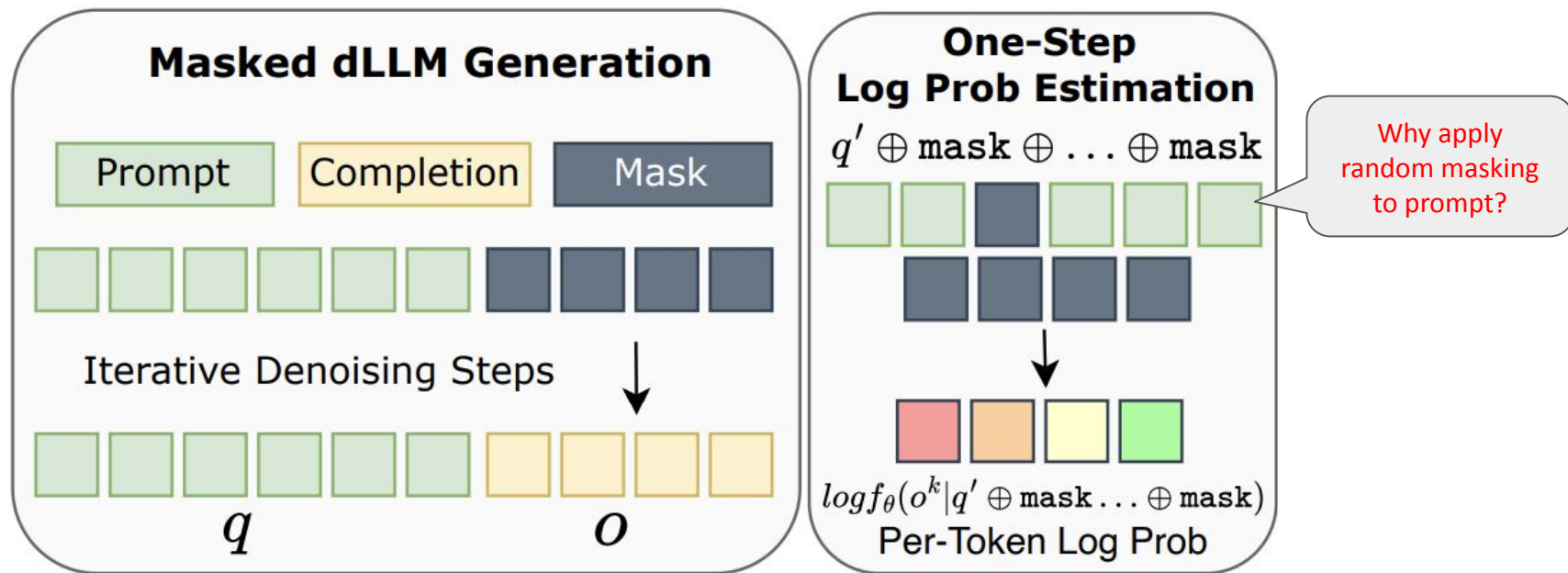
Algorithm 3 Conditional Log-likelihood Evaluation of LLaDA

Require: mask predictor p_θ , prompt p_0 , response r_0 , the number of Monte Carlo estimations n_{mc}

```
1: log_likelihood = 0
2: for  $i \leftarrow 1$  to  $n_{mc}$  do
3:    $l \sim \{1, 2, \dots, L\}$  #  $L$  is the sequence length of  $r_0$ 
4:   Obtain  $r_l$  by uniformly sampling  $l$  tokens from  $r_0$  without replacement for masking
5:    $\text{log\_likelihood} = \text{log\_likelihood} + \frac{L}{l} \sum_{i=1}^L \mathbf{1}[r_l^i = \mathbf{M}] \log p_\theta(r_0^i | p_0, r_l)$ 
6: end for
7:  $\text{log\_likelihood} = \text{log\_likelihood} / n_{mc}$ 
8: Return log_likelihood
```

- They find MC sample size of 128 to be stable.
- This estimator is inefficient for online RL:
 - it creates a large computational graph with hundreds of forward passes => excessive memory usage.

Efficient one-step log prob estimation



Generation Reuse during Policy Gradient Optimization

- ❑ One main bottleneck of GRPO is generation time, same for other online methods.
- ❑ A key way to make GRPO more sample-efficient is to reuse generated samples during optimization. This was actually introduced a long time ago with PPO.

Algorithm 1 Iterative Group Relative Policy Optimization

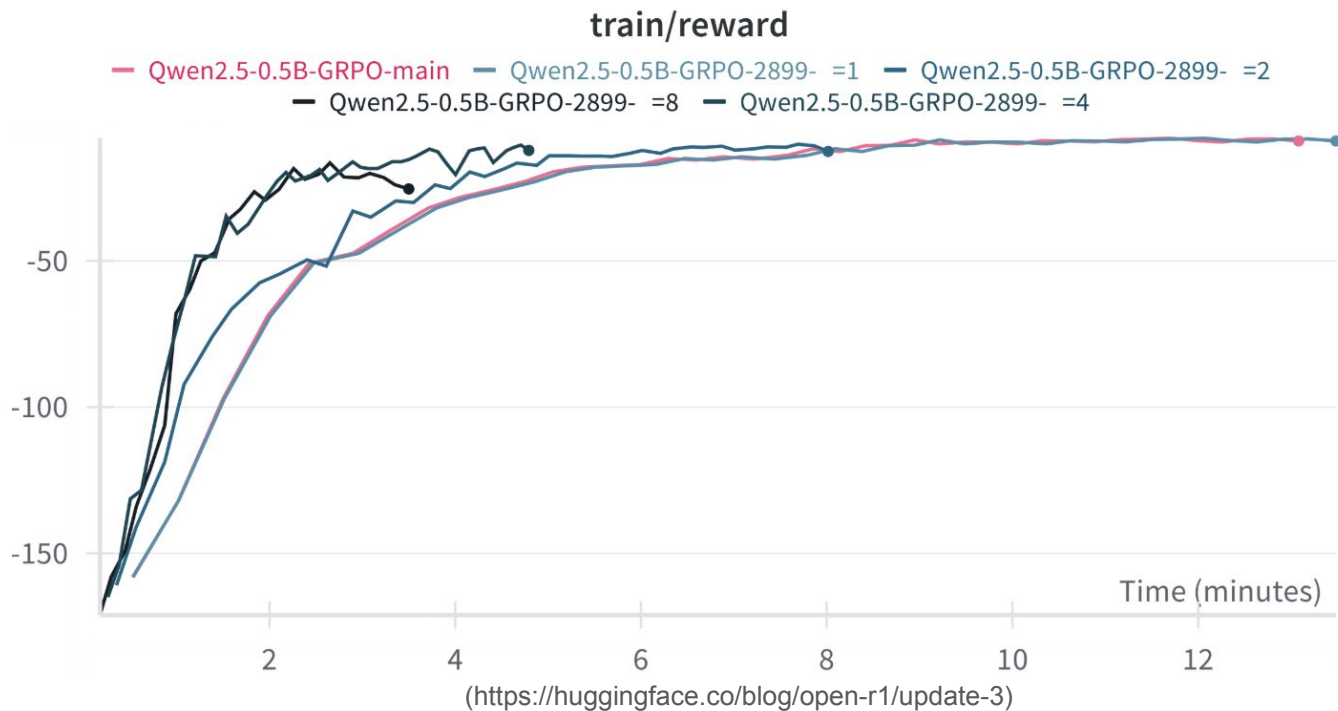
Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_{ϕ} ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

```
1: policy model  $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for iteration = 1, ..., I do
3:   reference model  $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ 
4:   for step = 1, ..., M do
5:     Sample a batch  $\mathcal{D}_b$  from  $\mathcal{D}$ 
6:     Update the old policy model  $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ 
7:     Sample  $G$  outputs  $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$  for each question  $q \in \mathcal{D}_b$ 
8:     Compute rewards  $\{r_i\}_{i=1}^G$  for each sampled output  $o_i$  by running  $r_{\phi}$ 
9:     Compute  $\hat{A}_{i,t}$  for the  $t$ -th token of  $o_i$  through group relative advantage estimation.
10:    for GRPO iteration = 1, ...,  $\mu$  do
11:      Update the policy model  $\pi_{\theta}$  by maximizing the GRPO objective (Equation 21)
12:  Update  $r_{\phi}$  through continuous training using a replay mechanism.
```

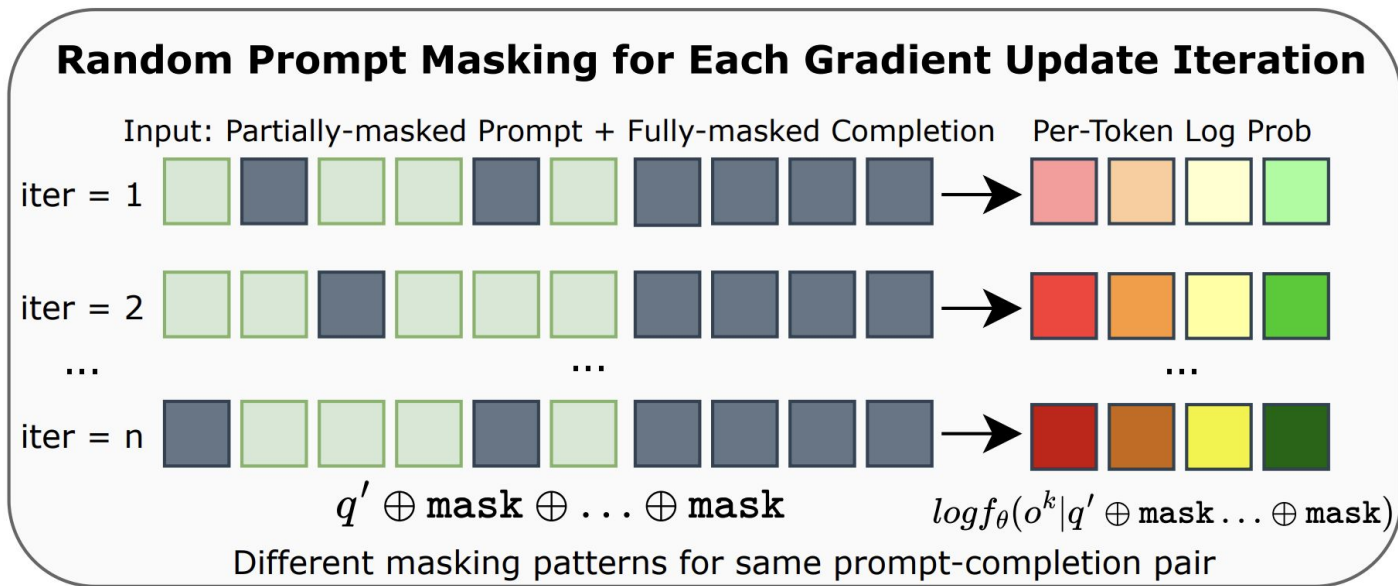
Output π_{θ}

Generation Reuse during Policy Gradient Optimization

- ❑ In practice, scaling μ too large will impact learning, due to overfitting.
- ❑ In TRL's open-r1 update, they suggested using $\mu=2$ or 4 for Qwen2.5

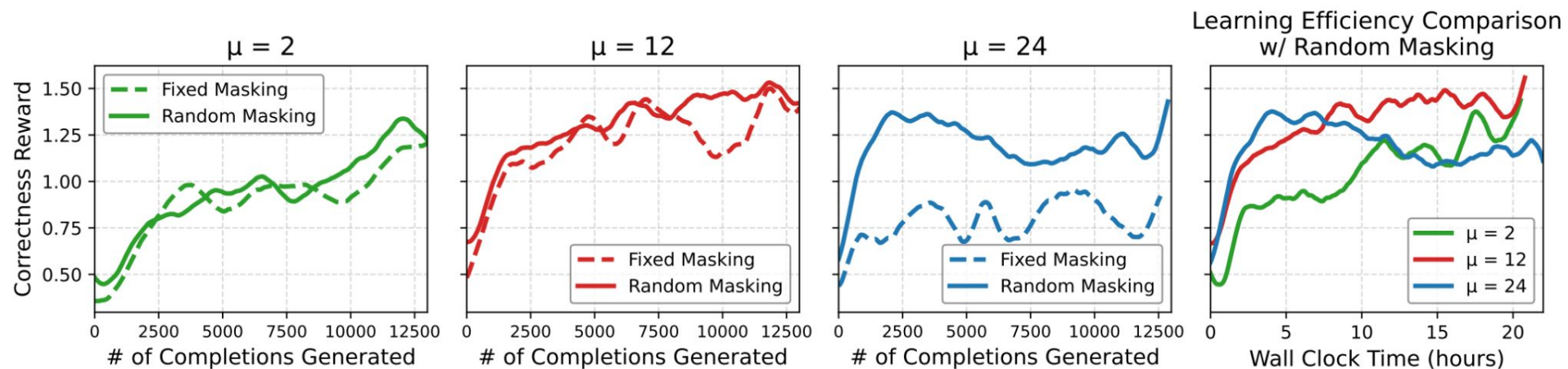


Random prompt masking mitigates this overfitting issue



Applying different random masks to prompt, creating diverse input of the **same prompt + completion pair** at each iteration.

Random prompt masking allows scaling to larger mu



Proposed framework to scale dLLMs reasoning: d1

Stage 1: Instilling reasoning behavior through SFT.

SFT on high-quality reasoning dataset.

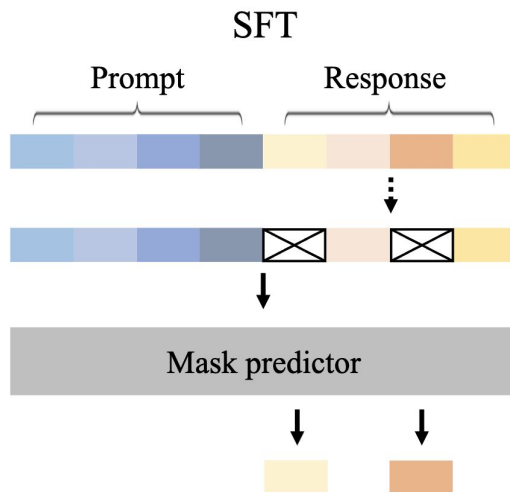
Stage 2: Reinforcing good reasoning traces via RL.

Apply diffu-GRPO on the SFT model.

We instantiate d1 using LLaDA-8B-Instruct as our base model

Stage 1: Masked SFT on s1K Dataset

- ❑ **Dataset: s1K (Muennighoff et al., 2025)**
 - ❑ 1,000 high-quality reasoning questions
 - ❑ Detailed step-by-step problem-solving traces
 - ❑ Includes verification and backtracking mechanisms



Stage 2: DiffuGRPO on 4 reasoning tasks

- ❑ Train on each tasks individually:
 - ❑ Math reasoning: GSM8K, MATH500
 - ❑ Logical reasoning: Countdown, 4x4 Sudoku Puzzles
- ❑ Reward Functions:
 - ❑ Rule-Based reward.
 - ❑ Correctness
 - ❑ Format

Results

Masked SFT on reasoning traces in s1k instill LLaDA with self-correction and backtracking behaviors.

Question: Jennifer's dog has 8 puppies 3 of which have spots. Brandon's dog has 12 puppies 4 of which have spots. What percentage of all the puppies have spots?

<reasoning> To determine the percentage of all the puppies that have spots, we need to follow these steps:

1. Calculate the total number of puppies from both dogs.
2. Calculate the total number of puppies with spots from both dogs.
3. Determine the percentage of puppies with spots.

First, let's find the total number of puppies from both dogs:

- Jennifer's dog has 8 puppies.
- Brandon's dog has 12 puppies.
- Total number of puppies = $8 + 12 = 20$.

Next, let's find the total number of puppies with spots from both dogs:

- Jennifer's dog has 3 puppies with spots.
- Brandon's dog has 4 puppies with spots.
- Total number of puppies with spots = $3 + 4 = 7$.

Now, we need to determine the percentage of puppies with spots. The formula for percentage is:

$$\text{Percentage} = \left(\frac{\text{Number of puppies with spots}}{\text{Total number of puppies}} \right) \times 100$$

Substituting the values we have:

$$\text{Percentage} = \left(\frac{7}{20} \right) \times 100$$

To simplify the fraction, we find the greatest common divisor (GCD) of 7 and 20, which is 1.

$$\frac{7}{20} = 7 \div 20$$

However, it seems there was a mistake in the calculation. Let's recheck the steps.

1. Total number of puppies = $8 + 12 = 20$.
2. Total number of puppies with spots = $3 + 4 = 7$.
3. Percentage of puppies with spots = $\left(\frac{7}{20} \right) \times 100$.

Let's calculate the percentage correctly:

$$\text{Percentage} = \left(\frac{7}{20} \right) \times 100 = \frac{7 \times 100}{20} = \frac{700}{20} = 35$$

The calculation answer. **< /reasoning><answer>** 35 **< /answer>**

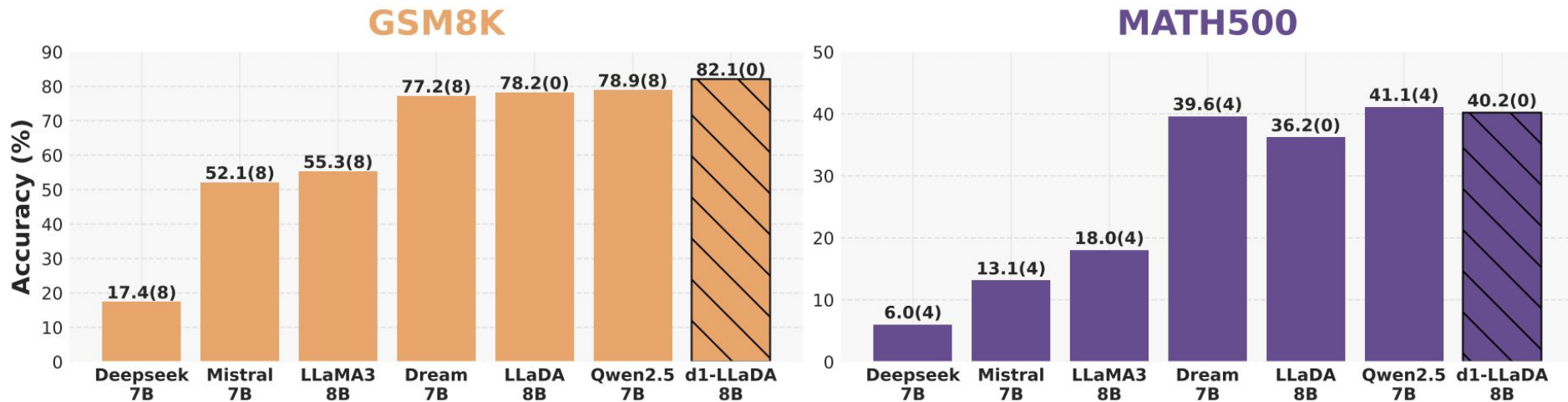
Results:

Table 1: **Model performance on GSM8K, MATH500, Countdown, and Sudoku Benchmarks:** All models are evaluated with 0-shot prompting, where the generation sequence length varies from 128 to 512. **Green values** indicate best performance and **blue values** indicate second-best performance in each column. The results demonstrate that **d1-LLaDA** consistently outperforms all other models, applying *diffu*-GRPO consistently improves the starting checkpoint, and *diffu*-GRPO alone shows better performance than SFT.

Model / Seq Len	GSM8K (0-shot)			MATH500 (0-shot)			Countdown (0-shot)			Sudoku (0-shot)		
	128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	68.7	76.7	78.2	26.0	32.4	36.2	20.7	19.5	16.0	11.7	6.7	5.5
+ SFT	66.5	78.8	81.1	26.2	32.6	34.8	20.3	14.5	23.8	16.5	8.5	4.6
+ diffu-GRPO	72.6	79.8	81.9	33.2	37.2	39.2	33.2	31.3	37.1	18.4	12.9	11.0
+ SFT + diffu-GRPO (d1-LLaDA)	73.2	81.1	82.1	33.8	38.6	40.2	34.8	32.0	42.2	22.1	16.7	9.5

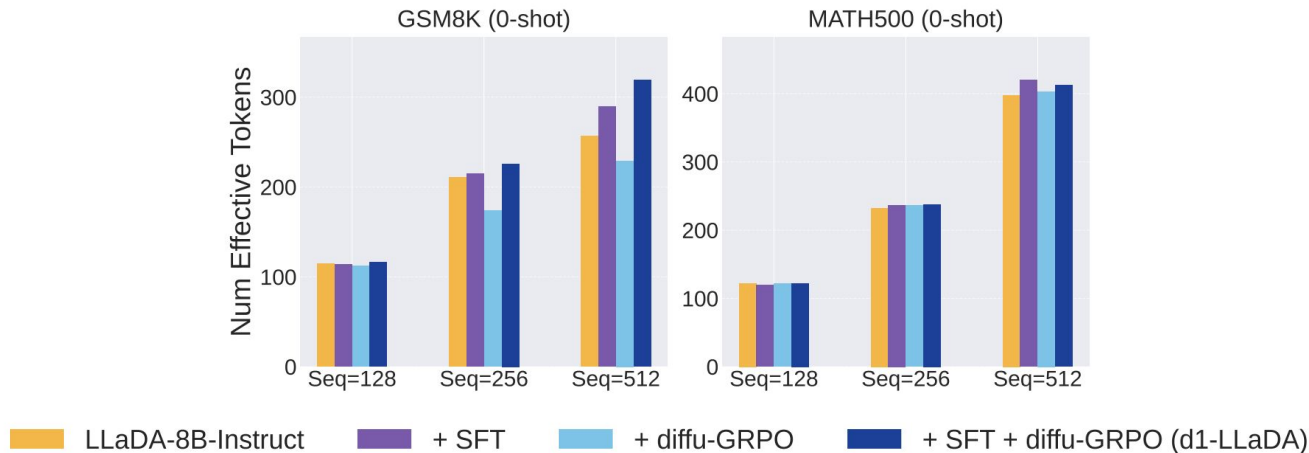
- ❑ diffu-GRPO consistently outperforms both base LLaDA and SFT
- ❑ diffu-GRPO consistently improve over their initialization checkpoint
- ❑ d1 recipe yields the highest gains
- ❑ diffu-GRPO improves reasoning beyond training sequence length (trained with fixed 256 seq len generation)

Comparison to SoTA dLLMs and similar sized AR LLMs



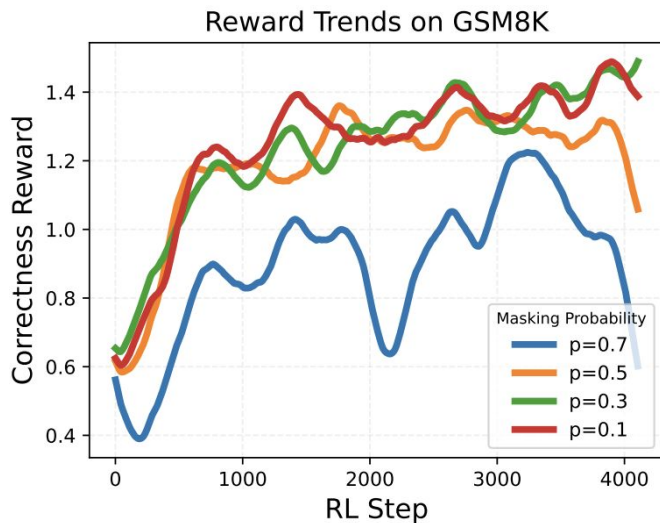
Effective Tokens Usage

- ❑ Effective tokens (# of non-padding, non-EOS tokens) grows with predefined sequence length.
- ❑ Applying SFT increase effective tokens.
- ❑ Applying RL alone decreases effective tokens.



Ablation on the prompt masking rate

Lower masking probabilities (0.1, 0.3) show more stable and higher performance, while higher masking probabilities (0.5, 0.7) demonstrate increased instability particularly in later training stages.



A better
masking
strategy?

Limitation & Future Work Directions

- ❑ **Fixed Generation Length:** LLaDA requires predetermined output length, limiting the model's ability to discover optimal reasoning paths, hindering emergence of advanced reasoning strategies.
 - ❑ Future Direction: Apply diffu-GRPO on Block Diffusion to enable flexible-length generation.
- ❑ **Task-Specific Training:** Currently, we train separate RL models for each reasoning task.
 - ❑ Future Direction: train a single dLLM on large reasoning datasets

Limitation & Future Work Directions

❑ **SFT Limitation:**

- ❑ LLaDA needs to take loss on the PAD tokens to learn to terminate its generation. It pads to the longest-seq in the batch, which means batch size will affect the training performance.
- ❑ We observe that training on truncated sequences will lead LLaDA to generate unfinished response.
- ❑ Future Direction: a more robust SFT algorithm.

Thank you!

Code: <https://github.com/dllm-reasoning/d1>

Paper: <https://arxiv.org/abs/2504.12216>

Website: <https://dllm-reasoning.github.io/>